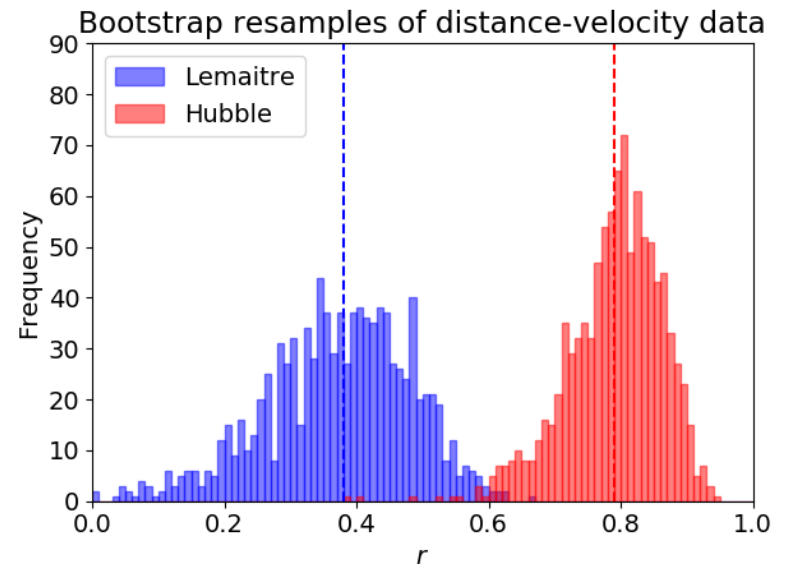
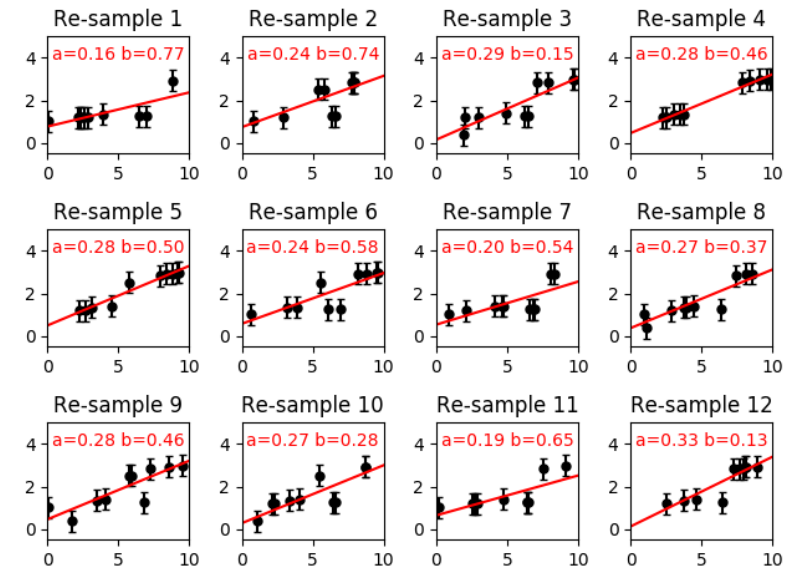
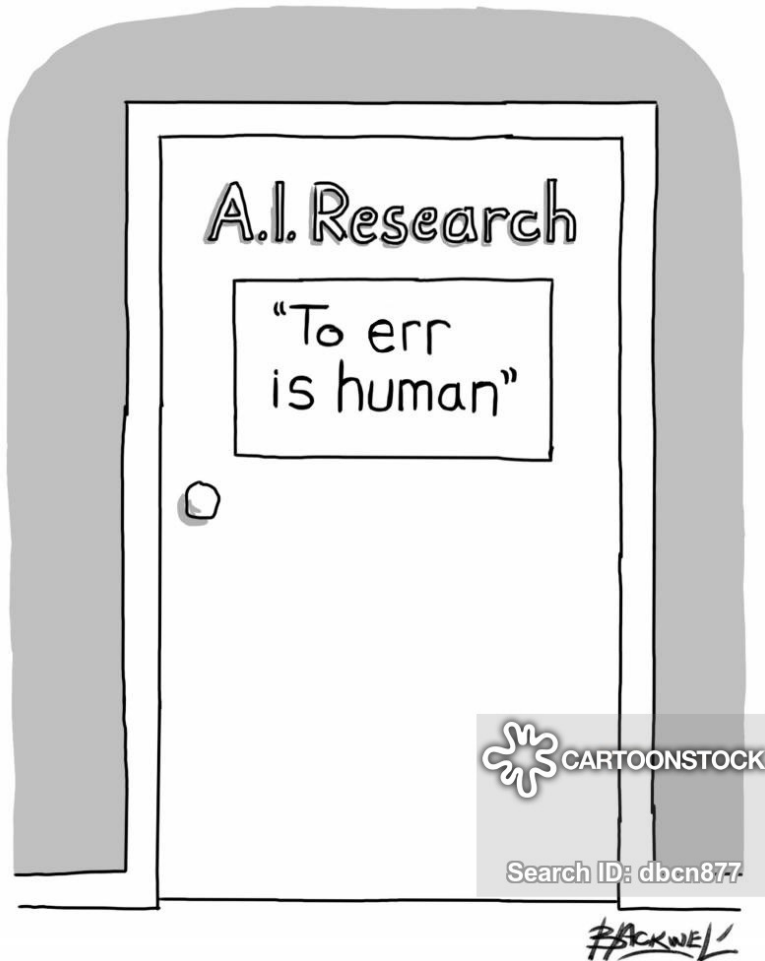
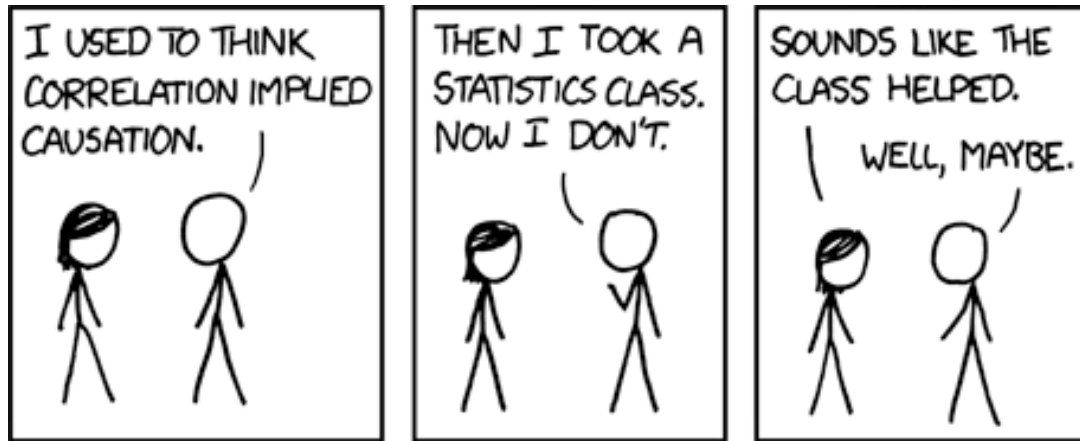


Class 5: Error Estimates



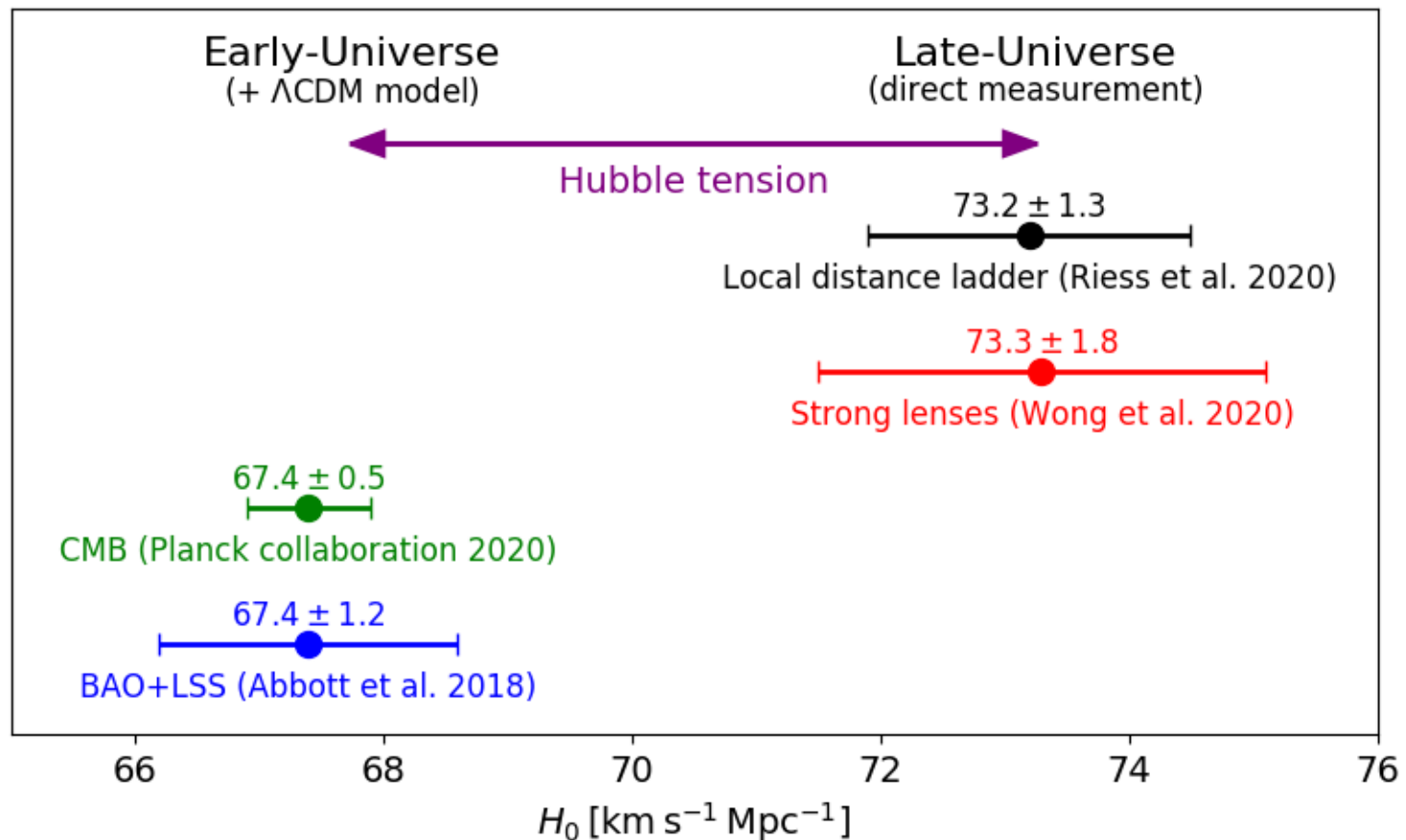
Class 5: Error Estimates



- Week 1: Probability and statistics
- Week 2: Correlation testing
- Week 3: Model fitting
- Week 4: Regression
- Week 5: Error estimates
- Week 6: Bayesian methods

What is an error?

- What does “ $H_0 = 73.2 \pm 1.3 \text{ km s}^{-1} \text{ Mpc}^{-1}$ ” mean?



Jack-knife errors

- Given a dataset with N entries $(x_1, x_2, x_3, \dots, x_N)$, we create N **separate datasets, deleting 1 entry in turn**

Dataset 1 – delete x_1 – $D_1 = (x_2, x_3, \dots, x_N)$

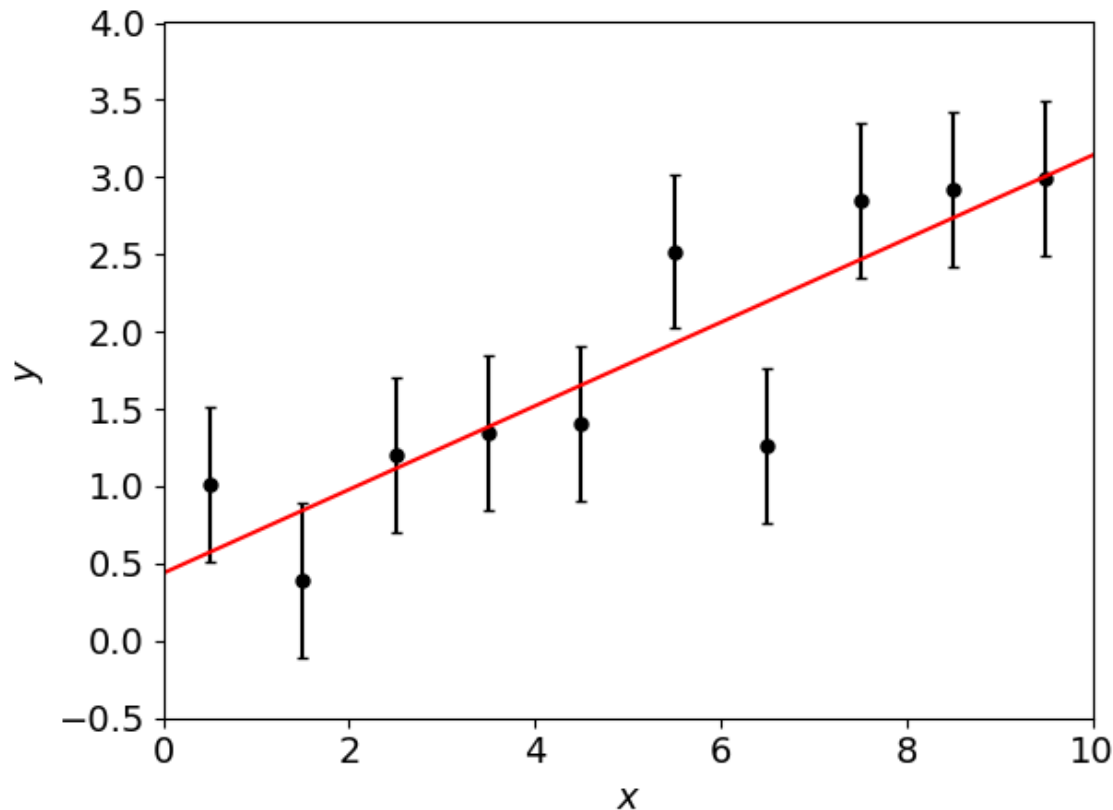
Dataset 2 – delete x_2 – $D_2 = (x_1, x_3, \dots, x_N)$

- We measure the statistic for each of these datasets $(D_1, D_2, \dots, D_N)$, creating N measurements $(S_1, S_2, \dots, S_N)$

- The error is given by: **JK error = $\sqrt{N - 1} \times \text{std dev of } S_i$**

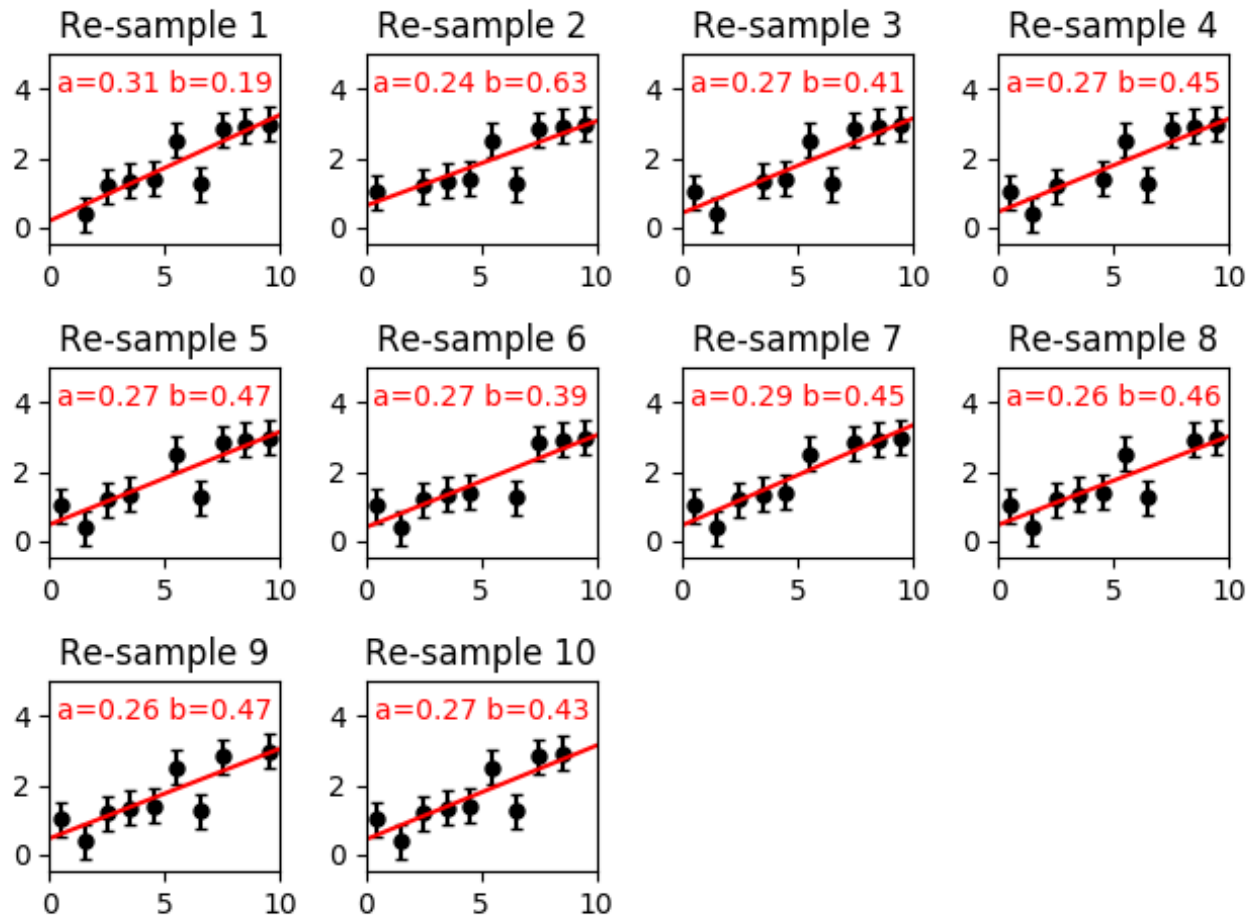
Worked example: Jack-knife errors

- Let's apply this procedure to fitting a straight line by minimizing the χ^2 statistic:



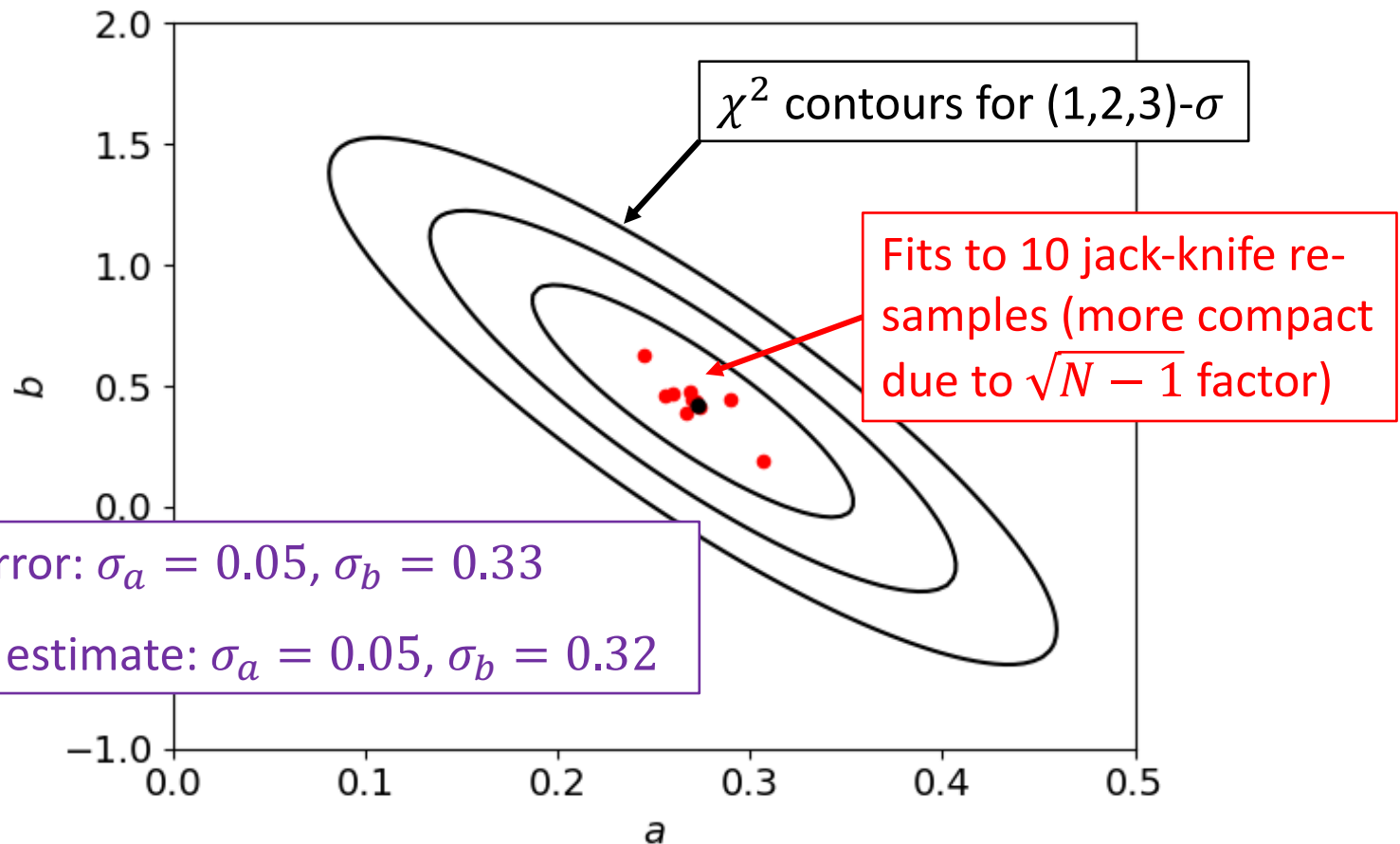
Worked example: Jack-knife errors

- Here are the fits to the 10 jack-knife samples deleting each point in turn:



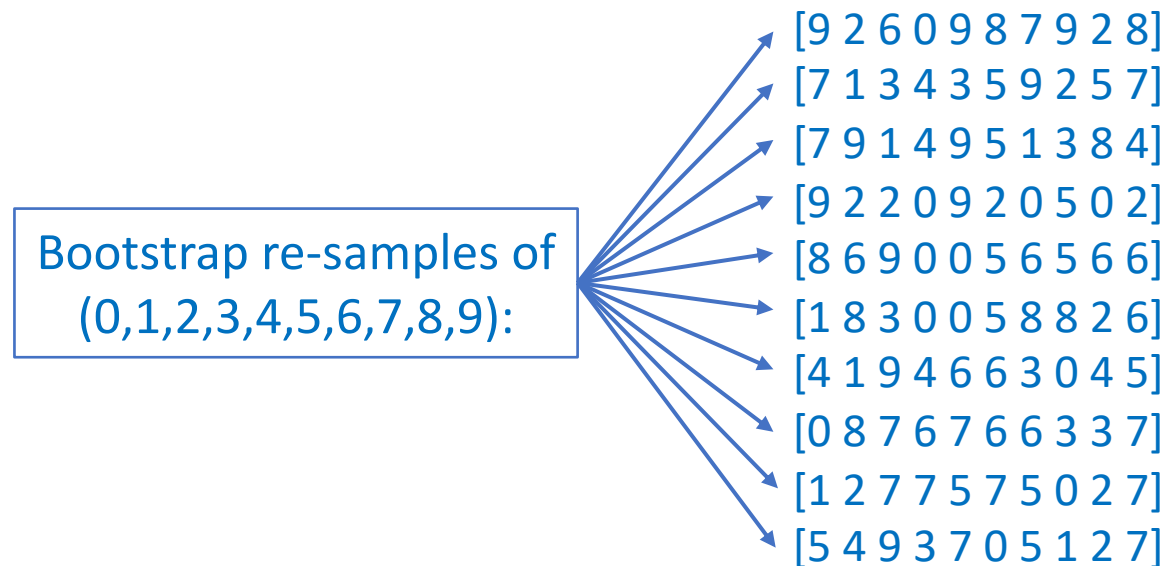
Worked example: Jack-knife errors

- Here are the best fits of (a, b) to each of those samples, compared to the original χ^2 contours:



Bootstrap errors

- If we have N data points, the procedure is to **repeatedly draw samples of N points at random, with replacement**
- *Hence, the same point can appear multiple times in each bootstrap sample!*

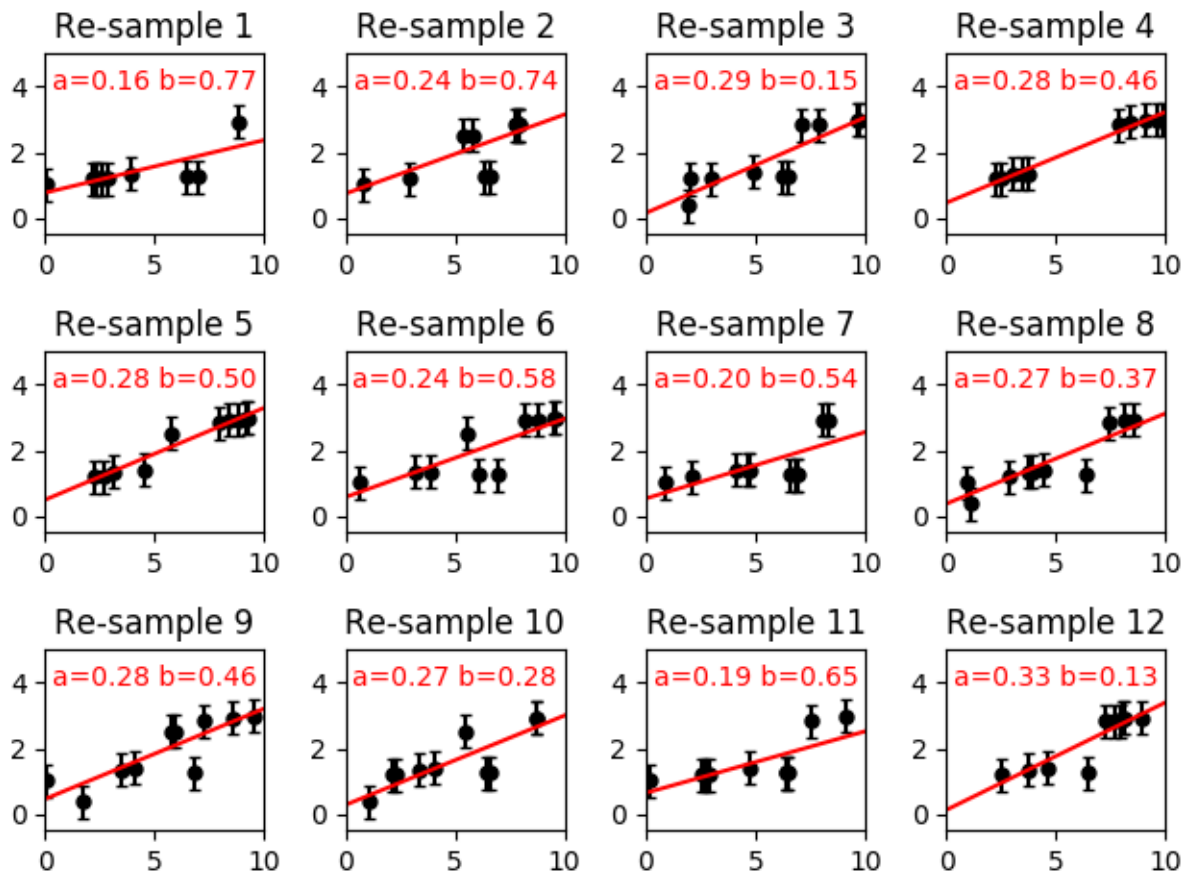


Bootstrap errors

- If we have N data points, the procedure is to **repeatedly draw samples of N points at random, with replacement**
- *Hence, the same point can appear multiple times in each bootstrap sample!*
- As with the jack-knife error, we measure the statistic for **each** of these bootstrap datasets (which can number $N_{\text{samp}} \gg N$ this time), creating measurements S_i
- The **bootstrap error = the standard deviation of S_i** (no extra scaling factors this time)

Worked example: Bootstrap errors

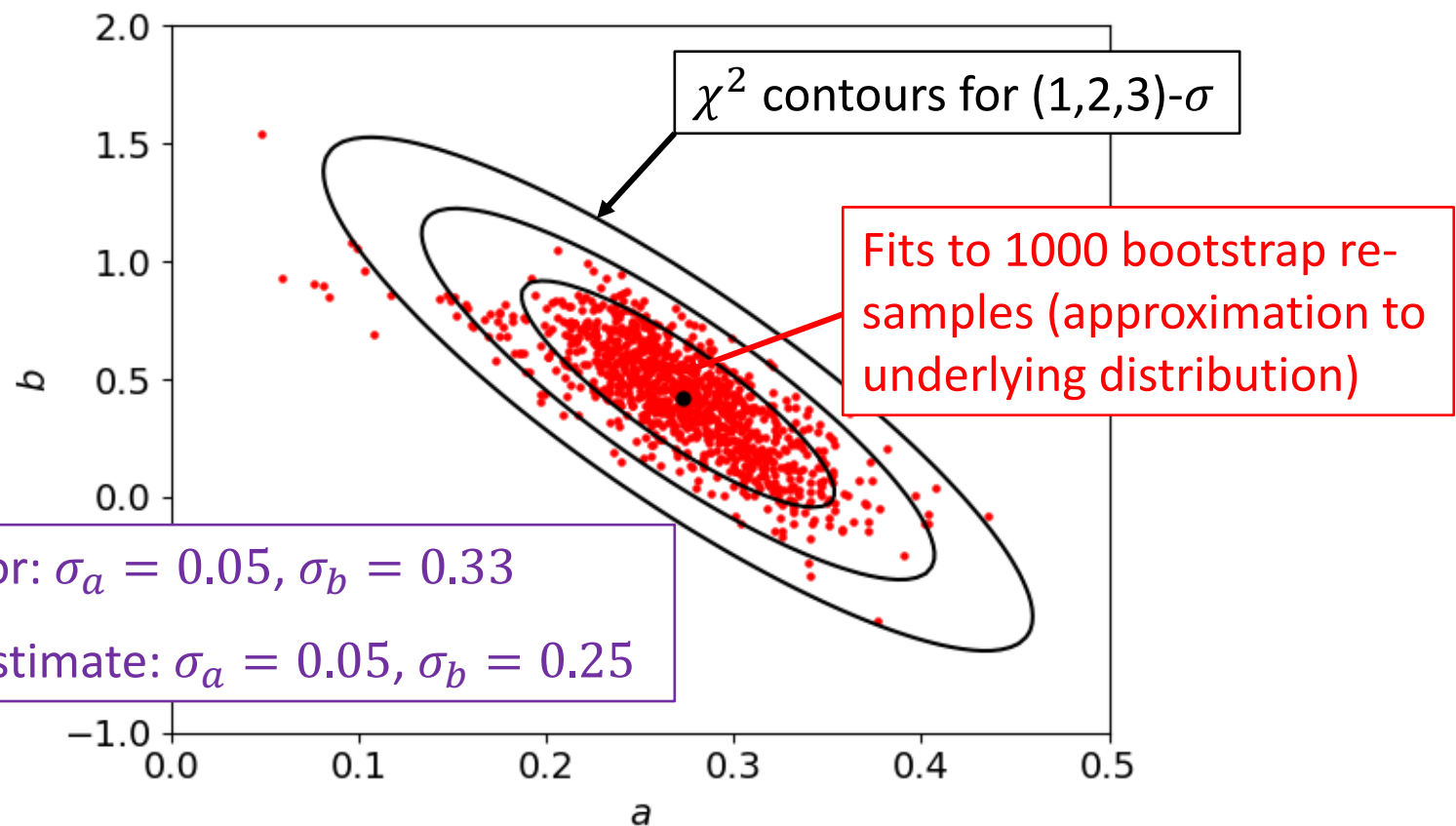
- Let's apply this approach to the straight-line fit. Here are the first 12 of 1000 bootstrap re-samples:



(I slightly offset the points, so you can see some are appearing more than once in each sample.)

Worked example: Bootstrap errors

- Here are the best fits of (a, b) for each of the 1000 bootstrap re-samples, compared to the original χ^2 contours:



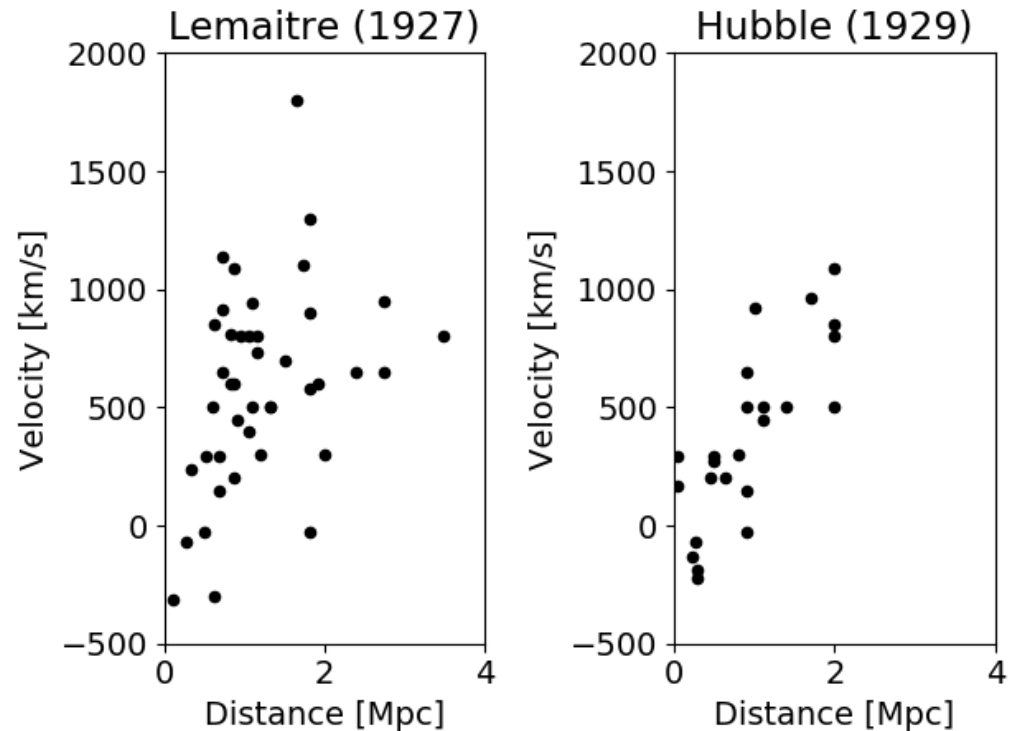
Original error: $\sigma_a = 0.05$, $\sigma_b = 0.33$

Bootstrap estimate: $\sigma_a = 0.05$, $\sigma_b = 0.25$

Activity: Bootstrap errors

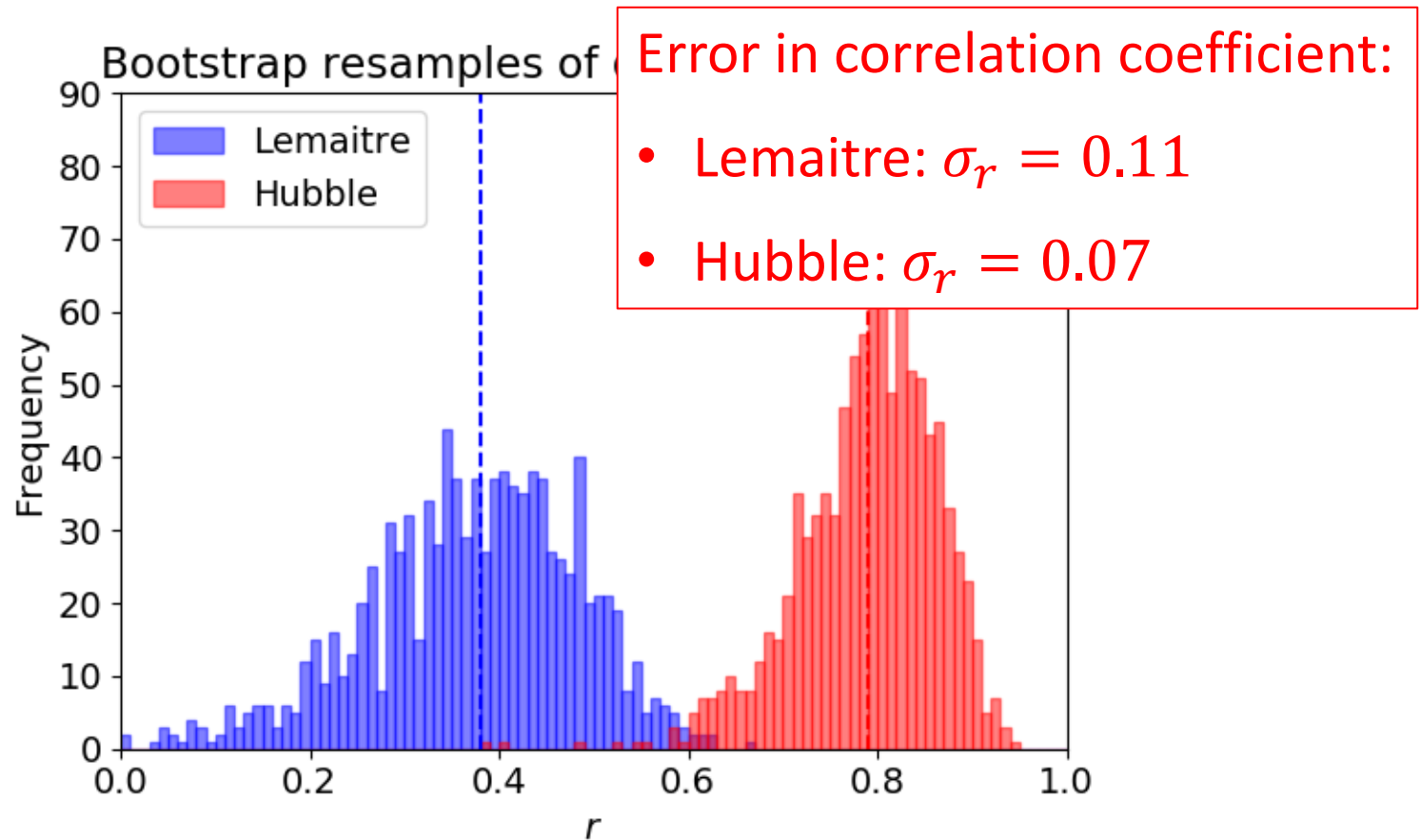
- In this Activity we will return to our previous analyses of Hubble and Lemaitre's distance-velocity datasets and determine **bootstrap errors** on our measurements

- Find the bootstrap error in the **correlation coefficient**
- Find the bootstrap error in the **slope H_0**



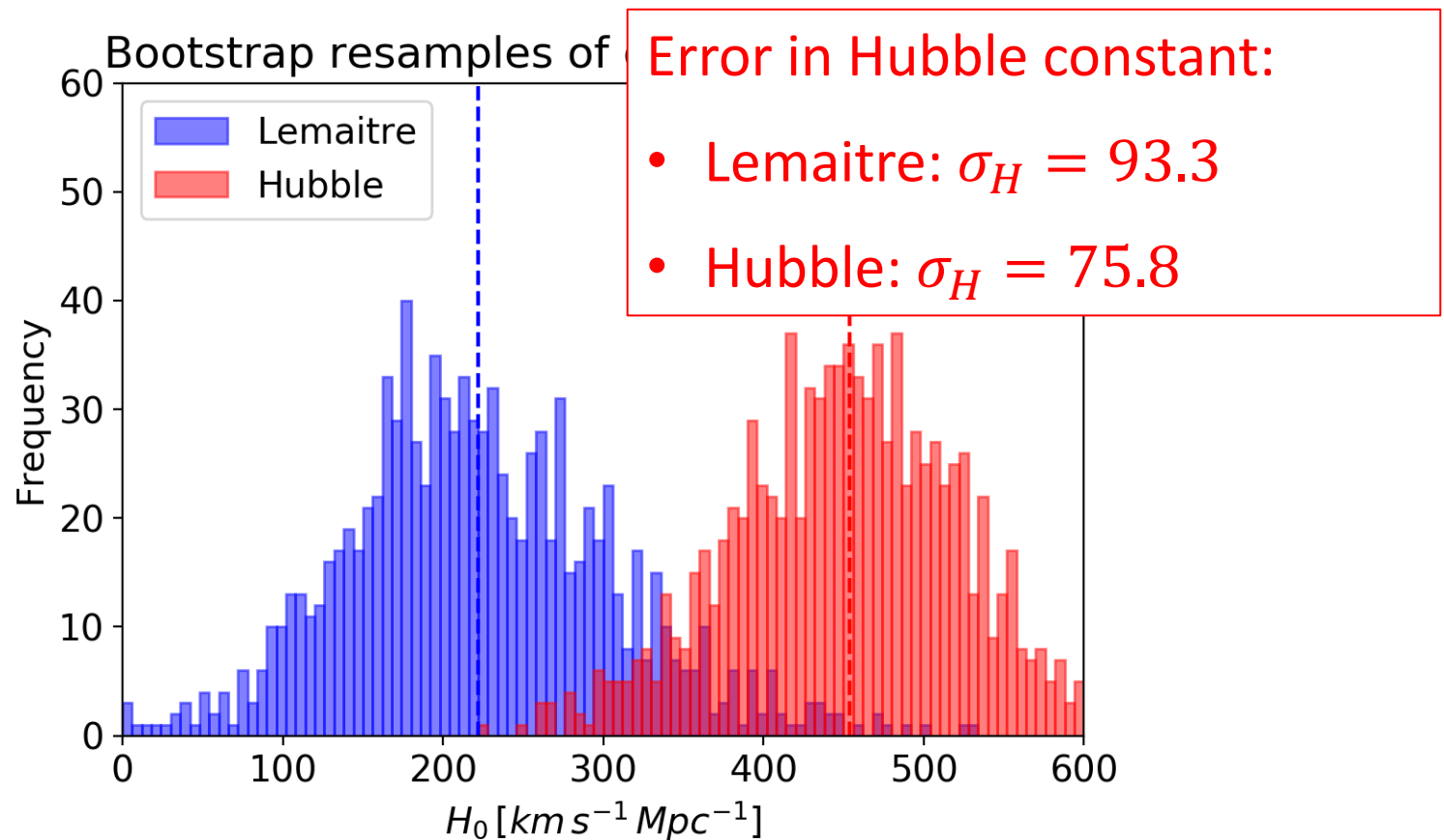
Activity: Bootstrap errors

- Bootstrap determination of the correlation coefficient errors:*



Activity: Bootstrap errors

- Bootstrap determination of the errors in the slope:*



Activity: Bootstrap errors

```
# Read in datasets
data = np.loadtxt('../Datasets/lemaitre.dat')
dl,vl = data[:,0],data[:,1]
data = np.loadtxt('../Datasets/hubble.dat')
dh,vh = data[:,0],data[:,1]
nl,nh = len(dl),len(dh)

# Generate bootstrap samples
nsamp = 1000
rlsave,rhsave = np.empty(nsamp),np.empty(nsamp)
alsave,ahsave = np.empty(nsamp),np.empty(nsamp)
for isamp in range(nsamp):
# Randomly select each bootstrap sample with replacement
    ind = np.random.choice(nl,nl,replace=True)
    dl1,vl1 = dl[ind],vl[ind]
    ind = np.random.choice(nh,nh,replace=True)
    dh1,vh1 = dh[ind],vh[ind]
# Correlation coefficient of each bootstrap sample
    rlsave[isamp],pl = stats.pearsonr(dl1,vl1)
    rhsave[isamp],ph = stats.pearsonr(dh1,vh1)
# Linear regression fit to each bootstrap sample
    result = stats.linregress(dl1,vl1)
    alsave[isamp] = result.slope
    result = stats.linregress(dh1,vh1)
    ahsave[isamp] = result.slope

# Error in correlation coefficient
print('Error in correlation coefficient for Lemaitre =',np.std(rlsave))
print('Error in correlation coefficient for Hubble =',np.std(rhsave))

# Error in slope (a.k.a. Hubble constant)
print('Error in slope for Lemaitre =',np.std(alsave))
print('Error in slope for Hubble =',np.std(ahsave))
```

Generate random sample
with replacement

Run fit on each sample

Standard
deviation
of result

Activity: Propagating errors

- A galaxy of absolute magnitude $M = -20$ is observed to have an apparent magnitude $m = 20.0 \pm 0.2$. What is the luminosity distance D_L in Mpc, and its error?
[Assume $m - M = 5 \log_{10} D_L + 25$]

My solution:

$$D_L = 10^{0.2m - 0.2M - 5} = 10^3 = 1000 \text{ Mpc}$$

$$\sigma_{D_L} = \left| \frac{\partial D_L}{\partial m} \right| \sigma_m = 0.2 D_L \ln 10 \sigma_m = 92 \text{ Mpc}$$

Activity: Propagating errors

- The total mass of a binary star system (in $M_{\odot}$) is given by Kepler's law $M = a^3 / P^2$, where a is the mean separation (in A.U.) and P is the period (in years). The α Centauri system has a period of $P = 79.9 \pm 1.0$ years and mean separation $a = 23.7 \pm 1.0$ A.U. **What is the total mass and error?**

My solution:

$$M = \frac{a^3}{P^2} = \frac{(23.7)^3}{(79.9)^2} = 2.09 M_{\odot}$$

$$\sigma_M^2 = \left(\frac{\partial M}{\partial a}\right)^2 \sigma_a^2 + \left(\frac{\partial M}{\partial P}\right)^2 \sigma_P^2 \rightarrow \left(\frac{\sigma_M}{M}\right)^2 = 9 \left(\frac{\sigma_a}{a}\right)^2 + 4 \left(\frac{\sigma_P}{P}\right)^2$$
$$\rightarrow \sigma_M = 0.27 M_{\odot}$$

Fisher matrix

- The **Fisher matrix** is a mathematical technique to propagate the statistical errors in a dataset, to the errors in the model parameters describing the dataset. The matrix is given by:

$$F_{ij} = \sum_k \frac{\partial m_k}{\partial p_i} \frac{1}{\sigma_k^2} \frac{\partial m_k}{\partial p_j}$$

i, j label the model parameters p_i , so the Fisher matrix dimension is $N_{par} \times N_{par}$

k labels the N data points d_k , so this sum is over N terms

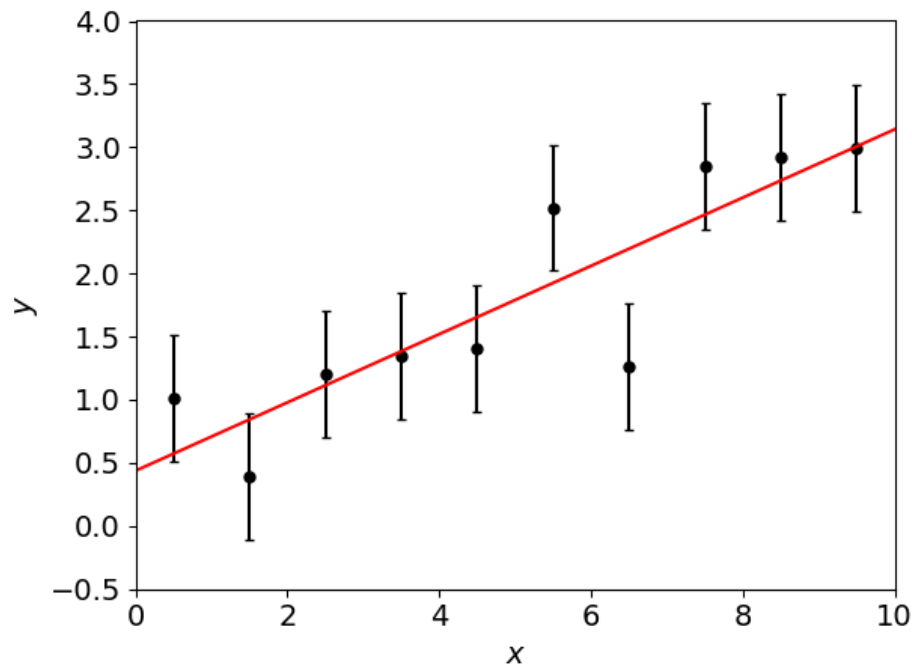
$\partial m_k / \partial p_i$ is the partial derivative of the model at point k , with respect to the parameter p_i

σ_k is the error in data point k

Worked example: Fisher matrix

$$F_{ij} = \sum_k \frac{\partial m_k}{\partial p_i} \frac{1}{\sigma_k^2} \frac{\partial m_k}{\partial p_j}$$

- Let's evaluate this for the example of **fitting a straight line**, $y = ax + b$ at N positions x_k



Worked example: Fisher matrix

$$F_{ij} = \sum_k \frac{\partial m_k}{\partial p_i} \frac{1}{\sigma_k^2} \frac{\partial m_k}{\partial p_j}$$

- Let's evaluate this for the example of **fitting a straight line**, $y = ax + b$ at N positions x_k
- There are 2 parameters, $p_i = (a, b)$
- The model is $m_k = ax_k + b$, so $\frac{\partial m_k}{\partial p_1} = x_k$ and $\frac{\partial m_k}{\partial p_2} = 1$

- The Fisher matrix is hence:
$$\begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix} = \begin{pmatrix} \sum_k \frac{x_k^2}{\sigma_k^2} & \sum_k \frac{x_k}{\sigma_k^2} \\ \sum_k \frac{x_k}{\sigma_k^2} & \sum_k \frac{1}{\sigma_k^2} \end{pmatrix}$$

Worked example: Fisher matrix

- We have $x_k = (0.5, 1.5, \dots, 9.5)$ and $\sigma_k = 0.5$, so we can evaluate the above matrix to $F = \begin{pmatrix} 1330 & 200 \\ 200 & 40 \end{pmatrix}$

- The **covariance matrix of the parameters is then given by the inverse of the Fisher matrix:**

$$\boxed{C = F^{-1}}$$

$$\begin{aligned} \text{Error in } p_1 &= \sqrt{C_{11}} \\ \text{Error in } p_2 &= \sqrt{C_{22}} \end{aligned}$$

- For our example, $C = \begin{pmatrix} 0.003 & -0.015 \\ -0.015 & 0.101 \end{pmatrix}$ hence the forecast errors are $\sigma_a = \sqrt{C_{11}} = 0.05$ and $\sigma_b = \sqrt{C_{22}} = 0.32$

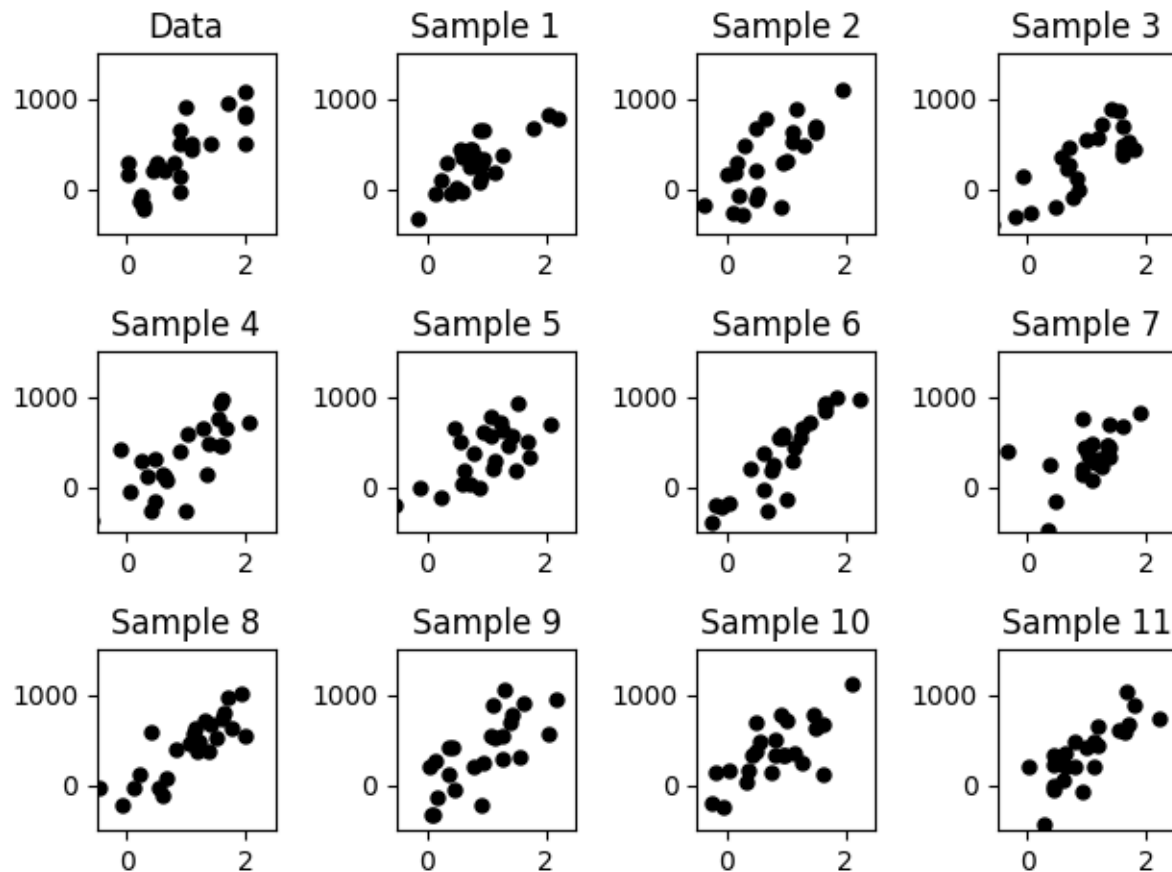
Monte Carlo simulation

- A **Monte Carlo simulation** is a computer model of an experiment in which many random realizations of the results are created and analysed like the real data
- This allows us to determine the errors in our measurements, as the *standard deviation of the fitted parameters over the realizations*



Activity: Monte Carlo simulation

- Run a **Monte Carlo simulation of Hubble's distance-redshift investigation**, and hence determine the error in H_0



Activity: Monte Carlo simulation

```
# Load in data
data = np.loadtxt('../Datasets/hubble.dat')
d,v = data[:,0],data[:,1]
n = len(d)

# We need to generate many realisations of this dataset with the same spread and correlation.
# There are many ways to do this, but I'll try generating realisations with the same mean, standard deviation
# and correlation coefficient.

# Find mean, standard deviation and correlation coefficient of data for generating Monte Carlo simulation
s = stats.describe(d)
dmu,dsig = s.mean,np.sqrt(s.variance)
s = stats.describe(v)
vmu,vsig = s.mean,np.sqrt(s.variance)
r,p = stats.pearsonr(d,v)

# The next section is slightly opaque probably ...
# I used the trick for converting 2 independent Gaussian variables into two correlated Gaussian variables
# (see Class 2 on correlation coefficient)
# I defined a new variable  $z = v/1000$  (probably no need to do so, but oh well!)
zsig = vsig/1000.
rho = r
r = rho*(zsig/dsig)
ysig = zsig*np.sqrt((1.-(rho**2))/(1.-(r**2)))

# Generate samples
nsamp = 1000
dsamp,vsamp = np.empty((nsamp,n)),np.empty((nsamp,n))
for isamp in range(nsamp):
    # Two independent Gaussian variables
    x = dsig*np.random.normal(size=n)
    y = ysig*np.random.normal(size=n)
    # Convert to two correlated Gaussian variables
    z = x*r + y*np.sqrt(1.-(r**2))
    dsamp[isamp,:] = dmu + x
    vsamp[isamp,:] = vmu + 1000.*z
```



Finding mean, standard deviation
and correlation coefficient of data

Generate datasets with
same correlations